

Westerlund: Westerlund ECM Panel Cointegration Test

Ho Ting (Bosco) Hung*

October 4, 2026

Abstract: This paper introduces a Python and R package, **Westerlund**, implementing the Westerlund (2007) panel cointegration tests. The tests determine whether a long-run relationship (cointegration) exists between a dependent variable y and a set of independent variables x across a panel of N cross-sectional units observed over T time periods. The Python and R implementations are designed to follow the estimation and bootstrap logic of the Stata command `xtwest` developed by Persyn and Westerlund (2008), while providing aligned user-facing functionality across both languages. In addition to the four Westerlund test statistics, the packages provide unit-level and mean-group results, reproducible bootstrap control, summary and reporting interfaces, and configurable visualization of the bootstrap distributions.

* Department of Politics and International Relations, University of Oxford

Contents

1	Overview	4
2	The Econometric Model	6
2.1	Hypothesis Testing Framework	7
3	R	8
3.1	Econometric Estimation Logic	10
3.1.1	Unit-Level Estimation	10
3.1.2	Long-Run Variance (HAC) Estimation	10
3.1.3	Panel Statistics Construction	11
3.1.3.1	Group Mean Statistics (<i>G</i>)	11
3.1.3.2	Pooled Panel Statistics (<i>P</i>)	11
3.2	Standardization and Display	11
3.3	Returned Results and Reporting	12
3.4	Bootstrap Inference	12
3.5	Visualization	14
3.6	Integration with the Other Packages	15
4	Python	17
4.1	Econometric Estimation Logic	19
4.1.1	Information Criterion and Model Selection	19
4.1.2	Long-Run Variance Estimation	19
4.1.3	The Error Correction Model (ECM)	19
4.1.4	Panel Statistics Construction	19
4.2	Bootstrap Inference	19
4.3	Standardization and Display	20
4.4	Returned Results and Reporting	20
4.5	Visualization	21

5	Points to Note	22
5.1	Randomization	22
5.2	OLS Equations	22
5.3	Floating Point Drifts	23
6	Implementation Examples	24
7	Conclusion	28
8	Bibliography	30

1 Overview

Econometric models, such as the Autoregressive Distributed Lag (ARDL) models, are increasingly used to analyze the long-run equilibrium relationship between a dependent variable y and a set of independent variables x across a panel of N cross-sectional units over T time periods (Pesaran & Shin, 1999; Pesaran et al., 2001). Before applying such models to the panel data, one has to first determine if a long-run equilibrium relationship exists between the variables using a cointegration test. However, panel data often suffer from the problem of cross-sectional dependence (CD/CSD), i.e., the units of observations are not independent of one another, and this is typically caused by unobserved common factors that affect all units (e.g., the global financial crisis in the case of investment flows) (Pesaran, 2021). Traditional residuals-based tests like the Pedroni panel cointegration test enforce strict exogeneity constraints and are not robust to cross-sectional dependence, so they are vulnerable to over-rejection (Pedroni, 2004). In cases of cross-sectional dependence, tests based on structural dynamics like the Westerlund (2007) test have to be used, which is now only available in Stata (Persyn & Westerlund, 2008).

This paper introduces Python and R implementations of the Westerlund (2007) tests, **Westerlund**, which can be accessed at <https://github.com/bosco-hung/WesterlundTest>. The Python version is also available at PyPI (<https://pypi.org/project/Westerlund/>) and the R version is available at CRAN (<https://cran.r-project.org/web/packages/Westerlund/index.html>). The two implementations are maintained with aligned public functionality and are designed to follow the logic of the Stata command `xtwest` developed by Persyn and Westerlund (2008). They calculate the four primary statistics (G_τ , G_α , P_τ , P_α), provide asymptotic standardization using the moments reported by Westerlund (2007), and implement a Stata-aligned residual bootstrap for cross-sectional dependence. Both versions also expose reusable result objects, unit-level and mean-group output, summary/reporting interfaces, reproducible bootstrap seeds, and configurable bootstrap visualization.

This paper starts by discussing the econometric model developed by Westerlund ([2007](#)). Then, it discusses the Python and R implementations. It proceeds with some points to note and closes with some implementation examples.

2 The Econometric Model

The Westerlund test is based on a structural equation known as the Conditional Error Correction Model (ECM):

$$\Delta y_{it} = \underbrace{\delta'_i d_t}_{\text{Deterministics}} + \underbrace{\alpha_i(y_{i,t-1} - \beta'_i x_{i,t-1})}_{\text{Long-Run Equilibrium}} + \underbrace{\sum_{j=1}^{p_i} \alpha_{ij} \Delta y_{i,t-j} + \sum_{j=-q_i}^{p_i} \gamma_{ij} \Delta x_{i,t-j}}_{\text{Short-Run Dynamics}} + e_{it} \quad (1)$$

where its core lies in the long-run equilibrium term $\alpha_i(y_{i,t-1} - \beta'_i x_{i,t-1})$.¹ The expression $y_{i,t-1} - \beta'_i x_{i,t-1}$ represents the deviation from equilibrium in the previous time period. If variables y and x are cointegrated, they move together in the long run according to the relationship $y_{it} = \beta_i x_{it}$. Therefore, the residual of $y_{i,t-1} - \beta'_i x_{i,t-1}$ represents the “error” or “disequilibrium” at time $t - 1$. The parameter α_i determines how the system responds to that disequilibrium. If $\alpha_i < 0$ (Error Correction), the system is stable. If y_{t-1} was “too high” relative to x_{t-1} (positive error), the negative α_i forces Δy_{it} to be negative. The variable y falls to restore equilibrium. On the other hand, if $\alpha_i = 0$ (No Error Correction), the change in y (Δy_{it}) does not depend on the previous level. The variables drift apart indiscriminately. This implies no cointegration.

The summation terms $\sum_{j=1}^{p_i} \phi_{ij} \Delta y_{i,t-j} + \sum_{j=-q_i}^{p_i} \gamma_{ij} \Delta x_{i,t-j}$ represent the short-run shocks. These terms $\sum_{j=1}^{p_i} \phi_{ij} \Delta y_{i,t-j}$ capture the inertia in the system. By including sufficient lags of Δy and Δx , we ensure that the regression residual e_{it} is free of serial correlation (white noise), so as to ensure the validity of the OLS t -statistics. On the other hand, the term $\sum_{j=-q_i}^0 \gamma_{ij} \Delta x_{i,t-j}$ includes current and future differences of x for correcting for endogeneity. By projecting the error onto the leads and lags of Δx , the regressors become strictly exogenous with respect to the error term, allowing for valid asymptotic inference on α_i .

The term $\delta'_i d_t$ accounts for trends in the data that are not related to the stochastic relationship between x and y . In Case 1 ($d_t = 0$), there is no intercept. The data has a

¹ $-\alpha_i \beta'_i$ is sometimes written as λ'_i .

zero mean; In Case 2 ($d_t = \{1\}$), there is a constant intercept. The data fluctuates around a non-zero level; Finally, in Case 3 ($d_t = \{1, t\}$), there is both an intercept and a linear trend. The data drifts upward or downward over time.

2.1 Hypothesis Testing Framework

The statistical test tests the value of α_i derived from the equation outlined in the following. Under the Null Hypothesis (H_0), there is no error correction, i.e. there is no cointegration exists for any unit:

$$H_0 : \alpha_i = 0 \quad \text{for all } i = 1 \dots N \quad (2)$$

The Alternative Hypothesis (H_1) depends on which statistic is used (Group vs. Panel): In the case of group mean statistics (G_τ, G_α), they do not assume a common speed of adjustment. They test whether cointegration exists for at least one unit in the panel:

$$H_1^G : \alpha_i < 0 \quad \text{for at least one } i \quad (3)$$

In the case of panel mean statistics (P_τ, P_α), they “pool” the information across the cross-sectional dimension and test whether cointegration exists for the entire panel as a whole, assuming a homogeneous speed of adjustment:

$$H_1^P : \alpha_i = \alpha < 0 \quad \text{for all } i \quad (4)$$

3 R

The R version of the code features a main driver: `westerlund_test`. This primary interface connects the functions to perform input validation and data sorting, as well as to handle the branching logic between asymptotic and bootstrap inference, which are summarized in Figure 1. It takes the following arguments:

```
1 westerlund_test(data,
2     yvar,
3     xvars,
4     idvar,
5     timevar,
6     constant = FALSE,
7     trend = FALSE,
8     lags = 1,          # Fixed value or c(min, max)
9     leads = NULL,      # Fixed value or c(min, max)
10    westerlund = FALSE,
11    aic = TRUE,
12    bootstrap = -1,     # <= 0 means no bootstrap
13    indiv.ecm = FALSE,
14    lrwindow = 2,
15    seed = NULL,
16    verbose = FALSE)
```

where `data` is the panel data input, `yvar` is a string specifying the dependent variable, and `xvars` is a character vector of regressors entering the long-run relationship. A maximum of six regressors is supported by the asymptotic moment tables; when `westerlund=TRUE`, the model is restricted to at most one regressor and requires at least a constant. The `idvar` and `timevar` arguments identify the panel and time dimensions. Inclusion of an intercept and linear trend is controlled by `constant` and `trend`. `trend=TRUE` requires `constant=TRUE`, and the trend is constructed within each unit as $1, \dots, T_i$, matching the Stata implementation. The short-run lag length p is defined by `lags`, which defaults to 1, while the lead length q is defined by `leads`, which defaults to 0 when `NULL`. A fixed integer specifies a common order, while a two-element range activates unit-specific

selection using AIC or BIC according to `aic`; with `westerlund=TRUE`, the Westerlund-specific information criterion is used. The `bootstrap` argument specifies the number of bootstrap replications, `indiv.ecm=TRUE` retains individual ECM regression tables, and `lrwindow` sets the Bartlett-kernel window. The optional integer `seed` makes the bootstrap reproducible within R; when `seed=NULL`, the current R random-number-generator state is used. Finally, `verbose` controls detailed console output.

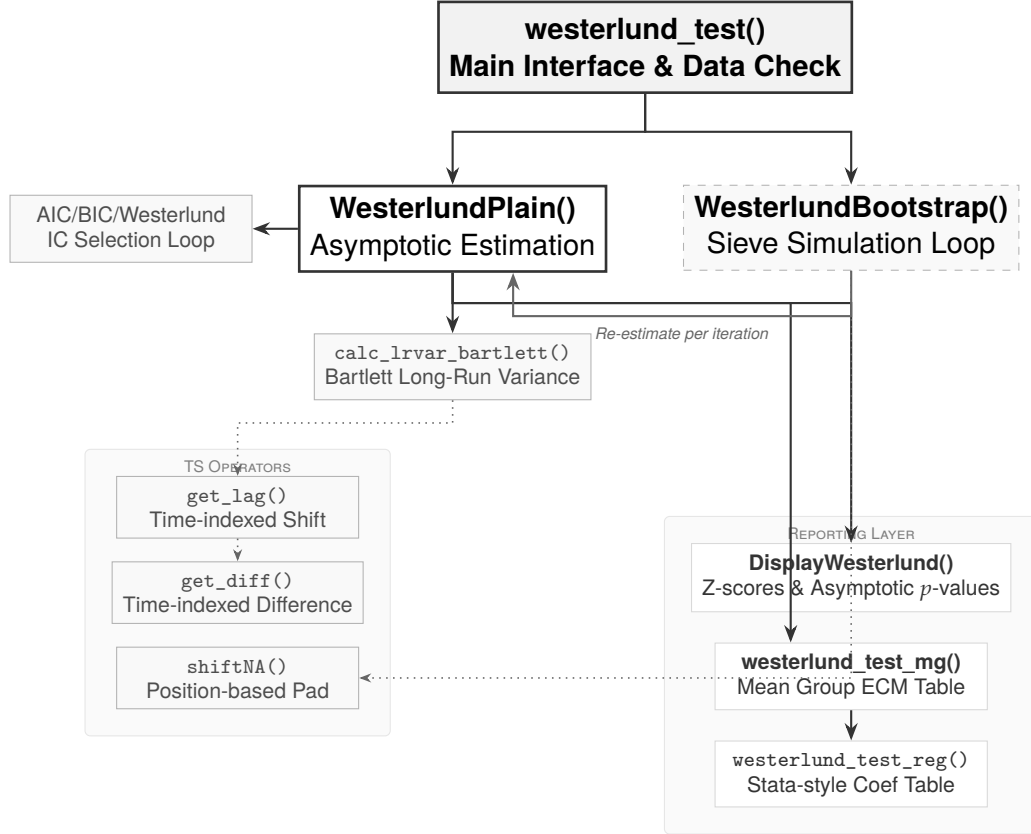


Figure 1: Structure of the R Version.

The implementation enforces strict time-series continuity by checking for time gaps:

$$\Delta t = t_{i,s} - t_{i,s-1} \quad (5)$$

If $\Delta t > 1$, a “hole” is identified, and the function halts to prevent invalid differencing.

The data is sorted to ensure the vector Y follows a block-unit structure:

$$Y = [y_{1,1}, y_{1,2}, \dots, y_{1,T}, \quad y_{2,1}, \dots, y_{2,T}, \quad \dots, \quad y_{N,T}]' \quad (6)$$

The implementation supports both balanced and unbalanced panels. Unit-specific panel lengths T_i may differ, including during bootstrap resampling. The requirement is that the time index remains continuous within each unit after observations with missing model variables are excluded. A balanced panel may nevertheless be empirically advantageous because the Westerlund ECM is parameter intensive and preserving observations improves precision, particularly in small macroeconomic panels. The workflow integration is discussed later in Section 3.6.

3.1 Econometric Estimation Logic

The `WesterlundPlain` function estimates ECMs and aggregates them into panel statistics.

3.1.1 Unit-Level Estimation

For each unit i , the code estimates the unrestricted ECM outlined in Equation 1. If `auto` selection is enabled, the code minimizes the Akaike information criterion (AIC) or the Bayesian information criterion (BIC). If `westerlund=TRUE`, it uses the specific penalty:

$$AIC(p, q) = \ln \left(\frac{RSS_i}{T_i - p - q - 1} \right) + \frac{2(p + q + \det + 1)}{T_i - p_{max} - q_{max}} \quad (7)$$

3.1.2 Long-Run Variance (HAC) Estimation

The helper `calc_lrvar_bartlett` computes the Newey-West variance using a Bartlett kernel (Newey & West, 1994). Following the Stata convention, autocovariances $\hat{\gamma}_j$ are scaled by $1/n$:

$$\hat{\omega}^2 = \hat{\gamma}_0 + 2 \sum_{j=1}^M \left(1 - \frac{j}{M+1} \right) \hat{\gamma}_j, \quad \hat{\gamma}_j = \frac{1}{n} \sum_{t=j+1}^n \hat{e}_t \hat{e}_{t-j} \quad (8)$$

3.1.3 Panel Statistics Construction

3.1.3.1 Group Mean Statistics (G) The group mean statistics (G) are computed by averaging individual unit results. Following the completion of the unit-level ECMs, the code aggregates the individual speed-of-adjustment coefficients ($\hat{\alpha}_i$) and their standard errors:

$$G_\tau = \frac{1}{N} \sum_{i=1}^N \frac{\hat{\alpha}_i}{SE(\hat{\alpha}_i)} \quad \text{and} \quad G_\alpha = \frac{1}{N} \sum_{i=1}^N \frac{T_i \hat{\alpha}_i}{\hat{\alpha}_i(1)} \quad (9)$$

where $\hat{\alpha}_i(1) = \frac{\hat{\omega}_{ui}}{\hat{\omega}_{yi}}$ is the ratio of long-run standard deviations derived from the Bartlett-kernel HAC estimation described earlier.

3.1.3.2 Pooled Panel Statistics (P) For pooled statistics (P_τ, P_α), the code partials out short-run dynamics and deterministics from Δy_{it} and $y_{i,t-1}$ using the average lag and lead orders calculated across all units. The pooled coefficient $\hat{\alpha}_{pooled}$ is then estimated by aggregating these filtered residuals:

$$\hat{\alpha}_{pooled} = \left(\sum_{i=1}^N \sum_{t=1}^T \tilde{y}_{i,t-1}^2 \right)^{-1} \sum_{i=1}^N \sum_{t=1}^T \frac{1}{\hat{\alpha}_i(1)} \tilde{y}_{i,t-1} \Delta \tilde{y}_{it} \quad (10)$$

The final statistics are then constructed as:

$$P_\tau = \frac{\hat{\alpha}_{pooled}}{SE(\hat{\alpha}_{pooled})} \quad \text{and} \quad P_\alpha = T \cdot \hat{\alpha}_{pooled} \quad (11)$$

where T represents the effective sample size after accounting for the loss of observations due to the chosen lag and lead lengths.

3.2 Standardization and Display

The function `DisplayWesterlund` converts raw statistics S into Z-scores using asymptotic moments μ_S and σ_S :

$$Z_S = \frac{\sqrt{N}(S - \mu_S)}{\sigma_S} \quad (12)$$

The moments are retrieved from hard-coded matrices indexed by deterministic cases (1: None, 2: Constant, 3: Trend) and the number of regressors K in the original Westerlund (2007) paper.

As the test is lower-tailed, the null of no cointegration is rejected if the observed statistic is significantly negative.

3.3 Returned Results and Reporting

The R function returns an object of class `westerlund_test`. In addition to the four raw statistics, the result object contains standardized Z-scores, asymptotic p -values, bootstrap p -values and bootstrap distributions when requested, unit-level estimates, detailed individual results, mean-group coefficients and standard errors, model settings, metadata, and reporting tables. The public `print()`, `summary()`, and `plot()` methods provide compact reporting, an extended programmatic summary, and bootstrap visualization respectively. When `indiv.ecm=TRUE`, unit-specific ECM coefficient tables are retained. Individual OLS tables use residual degrees of freedom and t inference, while the mean-group reporting tables use large-sample normal (z) inference.

3.4 Bootstrap Inference

The R implementation handles cross-sectional dependence through a residual bootstrap conducted under the null hypothesis $H_0 : \alpha_i = 0$. The procedure follows the following bootstrap logic:

1. **Restricted Null-Model Estimation:** For each unit i , the short-run model is estimated under the null, excluding the lagged level terms that generate error correction. When lag or lead ranges are supplied, the optimal orders are selected from this restricted model. Residuals $\hat{e}_{it}$ are centered within each unit. The differenced regressors Δx_{it} are also centered within unit, with their means calculated over all usable observations rather than only the residual-estimation support.

2. **Actual-Time Cluster Resampling:** To preserve contemporaneous cross-sectional dependence, sampling is conducted on the actual time clusters rather than independently by unit or by within-unit row position. The implementation emulates the Stata sequence `expandcl 2, bsample, cluster(t)`, and the subsequent `newttt/tussent/newtt` alignment and stable sorting. Because the original T_i is retained for each unit, an unbalanced panel remains unbalanced in each bootstrap replication.

3. **Innovation Construction (u_{it}^*):** The bootstrap innovation combines the resampled residual with the lagged, contemporaneous, and led centered regressor differences:

$$u_{it}^* = e_{it}^* + \sum_{j=-q}^p \hat{\gamma}_{ij} \Delta x_{i,t-j}^* \quad (13)$$

The `shiftNA` helper pads out-of-range lags and leads with `NA`, and boundary missings are handled in the same reconstruction sequence used by the Stata-aligned bootstrap before the autoregressive recursion begins. For compatibility with the original 2010 `xtwest` source, the implementation also reproduces its `currlead` behavior: when automatically selected lead orders differ across units, the final processed unit's selected lead controls the global bootstrap lead loop, while unavailable unit-specific coefficients are zero.

4. **Recursive Simulation:** The series of first differences is generated recursively using the unit-specific autoregressive coefficients:

$$\Delta y_{it}^* = \sum_{j=1}^p \hat{\phi}_{ij} \Delta y_{i,t-j}^* + u_{it}^* \quad (14)$$

The initial lag periods form a burn-in region, and the reconstructed series is subsequently restricted using each unit's original T_i .

5. **Integration and Re-Estimation:** The simulated differences are accumulated to recover level variables:

$$y_{it}^* = \sum_{s=1}^t \Delta y_{is}^*, \quad X_{it}^* = \sum_{s=1}^t \Delta X_{is}^* \quad (15)$$

The resulting simulated panel is then passed back to `WesterlundPlain`, which recalculates G_τ , G_α , P_τ , and P_α . Repeating this procedure B times produces the empirical null distributions.

The package reports the finite-sample-corrected bootstrap p -value:

$$p^* = \frac{\sum_{b=1}^B I(Stat_b^* \leq Stat_{obs}) + 1}{B + 1} \quad (16)$$

3.5 Visualization

The R implementation provides the `plot.westerlund_test` S3 method, invoked through `plot(result)`, to visualize bootstrap inference. It uses the **ggplot2** framework to create a faceted 2×2 grid for G_τ , G_α , P_τ , and P_α . The lower-tail critical probability is configurable through `conf_level`; robust bootstrap p -values may be displayed through `show_robust_p`; and the figure can be exported directly through `save_path`, `dpi`, and `figsize`. Colors, line widths, density transparency, and grid display are also configurable.

Mathematically, given B bootstrap replications, the function estimates the empirical null distribution using a kernel density estimator:

$$\hat{f}(s) = \frac{1}{Bh} \sum_{b=1}^B K\left(\frac{s - Stat_b^*}{h}\right) \quad (17)$$

where $Stat_b^*$ represents the simulated statistics for $b = 1, \dots, B$, $K(\cdot)$ is the Gaussian kernel, and h is the bandwidth automatically determined by the `geom_density` geometry.

The visualization logic is structured as follows. The kernel density curve represents the empirical bootstrap null distribution; a solid vertical line marks the observed statistic $Stat_{obs}$; and a dashed vertical line marks the empirical lower-tail critical value CV^* at the

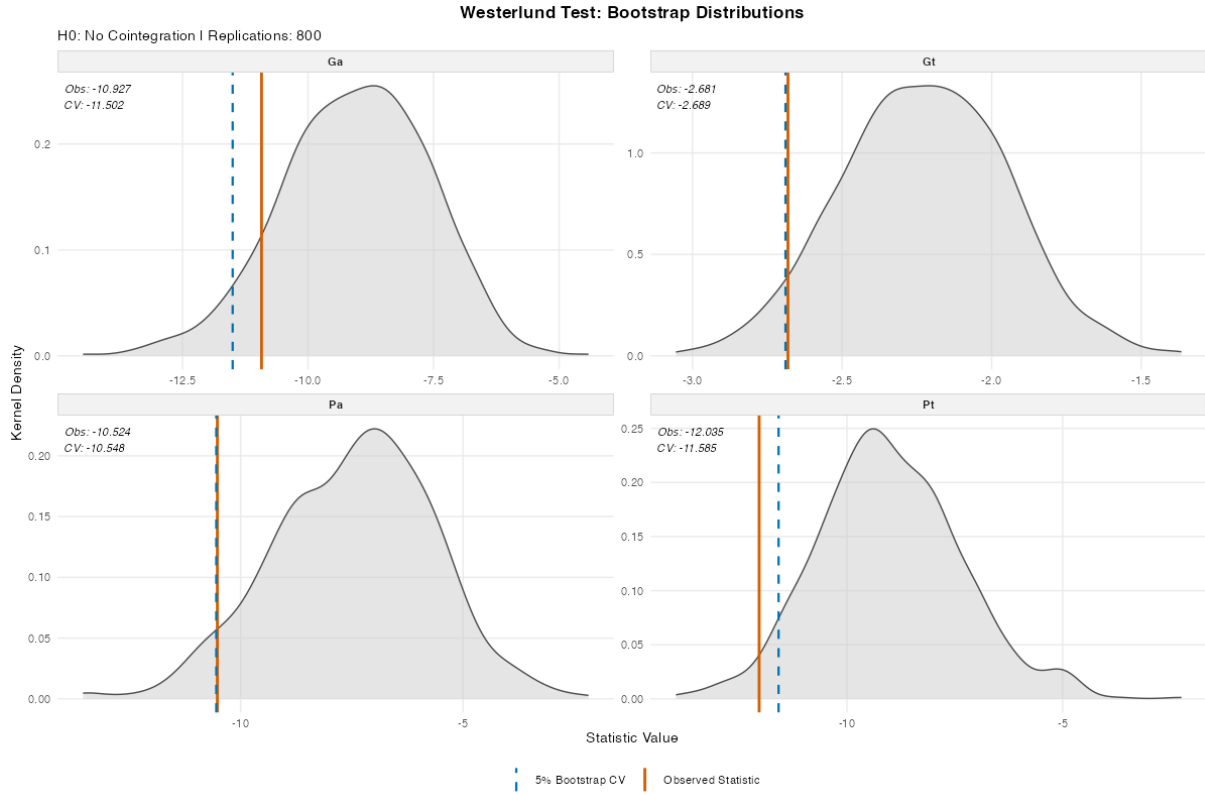


Figure 2: *Bootstrapping Distribution: R Replication of the First Table on p.240 in Persyn and Westerlund (2008).*

probability specified by `conf_level`. Each facet reports the observed value and critical value and, by default, the robust bootstrap p -value.

The null hypothesis is rejected at the α level if the observed statistic (solid line) lies to the *left* of the bootstrap critical value (dashed line). If the density mass is located significantly to the right of the observed statistic, the result is considered robust. Significant overlap between the density and the observed statistic suggests that asymptotic significance may be a false positive caused by cross-sectional dependence.

Figure 2 provides an example of the visualization result generated by replicating the first table on p.240 in Persyn and Westerlund (2008).

3.6 Integration with the Other Packages

The **Westerlund** package can be easily integrated into the workflow of users investigating the long-run relationship of variables in panel data. Examinations of long-run

dynamics often involve (1) testing the existence of cross-sectional dependence, (2) testing the order of integration, (3) testing the existence of cointegration, and finally (4) running the error-correction model tests. Regarding cross-sectional dependence, in the R ecosystem, the Pesaran cross-sectional dependence test is available (Pesaran, 2021) in `pcdtest` of the **plm** package (Croissant et al., 2006). Regarding the order of integration, the **plm** package provides several first-generation tests, including the Levin-Lin-Chu (LLC) (Levin et al., 2002), Im-Pesaran-Shin (IPS) (Im et al., 2003), and Hadri stationarity tests (Hadri, 2000). In case there is cross-sectional dependence, the **plm** package also offers the second-generation Cross-sectionally Augmented IPS (CIPS) test proposed by Pesaran (2007) which is robust to cross-sectional dependence. Finally, researchers can use **PooledMeanGroup** or **ARDL** to run mean group (MG) or panel mean group (PMG) ARDL models (Natsiopoulous & Tzeremes, 2020; Zientara & Kujawski, 2017). The **Westerlund** package introduced by this paper solves the roadblock regarding the test of the existence of cointegration, as summarized in Figure 3.

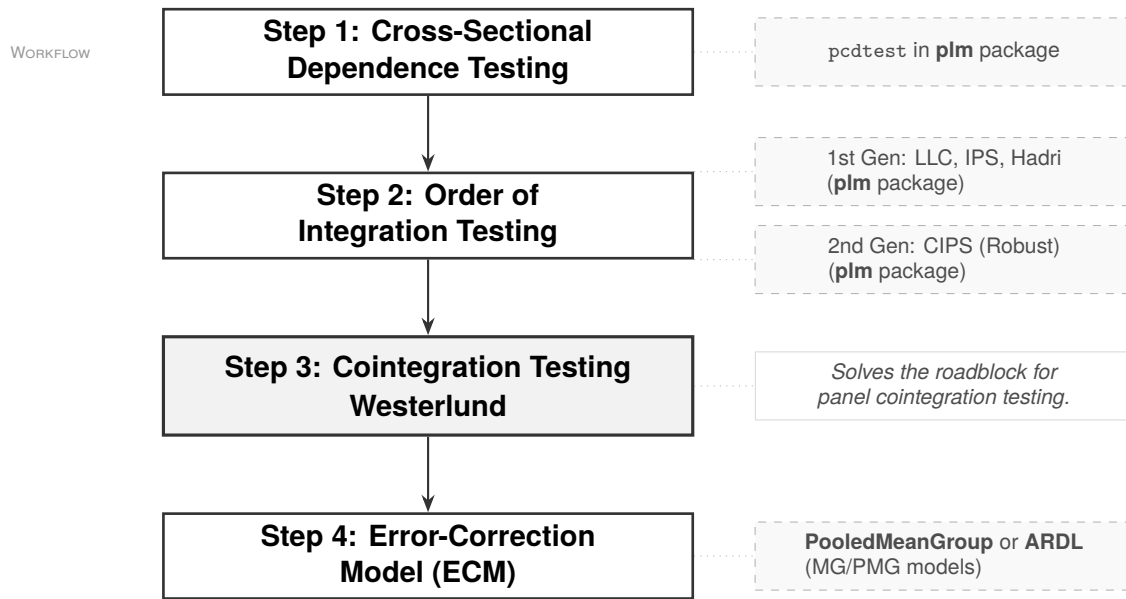


Figure 3: Workflow Integration.

4 Python

A Python version of the Westerlund Test framework is also developed to accommodate the preferences of different users. The `WesterlundTest` class provides a modular framework for evaluating cointegration in panel data and its structure is summarized in Figure 4. The class can be initialized as follows:

```
1 WesterlundTest(data, y_var, x_vars, id_var, time_var,
2               lags=1, leads=0, lrwindow=2, constant=False, trend=False,
3               westerlund=False, aic=True, bootstrap=-1, indiv_ecm=False,
4               seed=None, verbose=False)
```

The `data` parameter expects a `pandas.DataFrame`, with `y_var` identifying the dependent variable and `x_vars` identifying one or more long-run regressors. The panel and time dimensions are defined by `id_var` and `time_var`. The `lags` and `leads` arguments accept a fixed non-negative integer or a two-element range for unit-specific automatic selection; `lags` defaults to 1 and `leads` to 0. The Bartlett-kernel window is specified by `lrwindow`. The deterministic components are controlled by `constant` and `trend`; as in the R implementation, `trend=True` requires `constant=True`. The class enforces the same public restrictions as R: at most six regressors are supported by the asymptotic moment tables, while `westerlund=True` requires at least a constant and at most one regressor. The `aic` flag selects AIC rather than BIC for ordinary automatic selection, while the specialized Westerlund criterion is used when requested. The `bootstrap` argument specifies the number of bootstrap replications, `indiv_ecm=True` retains unit-specific ECM tables, and `seed` controls reproducibility without forcing a fixed seed when it is omitted. Finally, `verbose` controls detailed console output.

Similar to the R version, the Python method has a built-in gap-aware logic that does not rely on simple position-based shifting. The methods `_get_ts_map` and `_get_lag_lead` form the core of the gap-aware logic. The class maps values to their specific time index T . When a lag k is requested, the engine looks for the value at $T - k$. If the time index is discontinuous, the engine assigns a NA to that observation to prevent the model from erroneously calculating short-run dynamics across missing time periods.

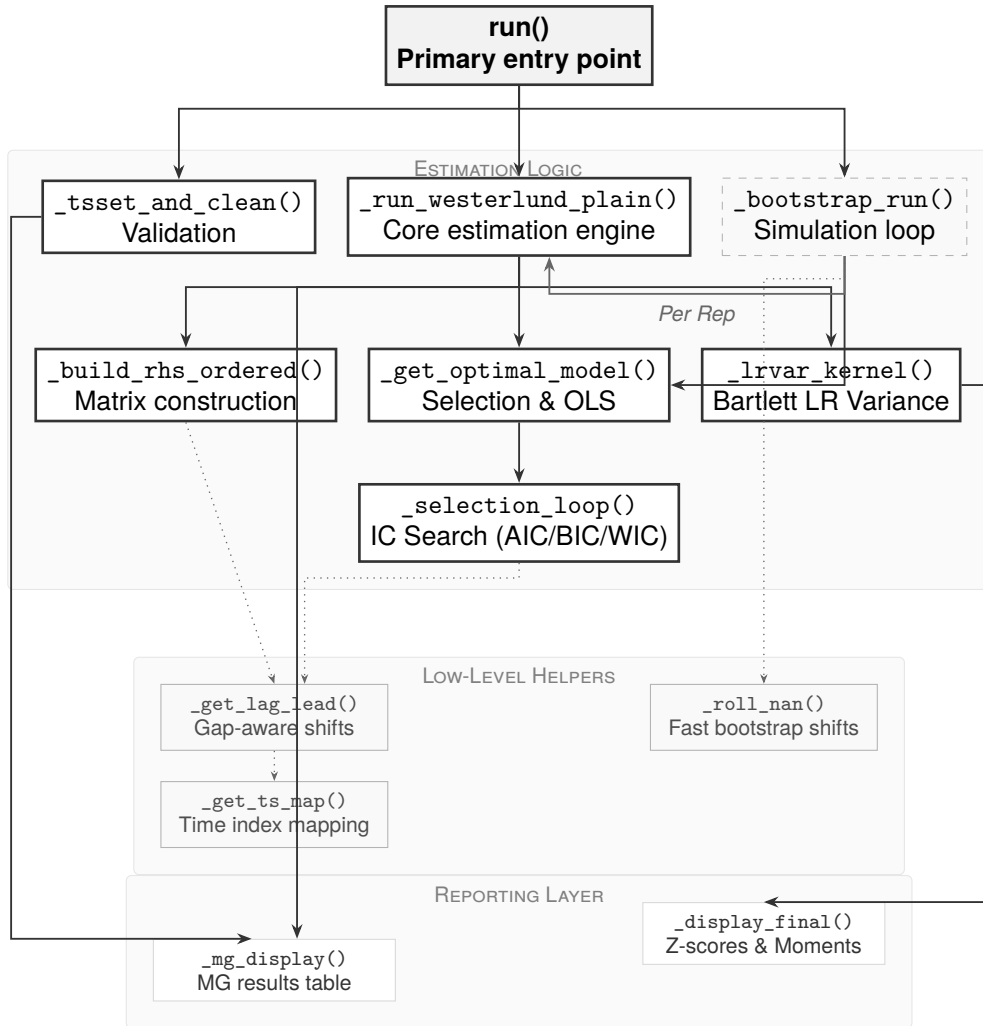


Figure 4: Structure of the Python Version.

4.1 Econometric Estimation Logic

4.1.1 Information Criterion and Model Selection

The `_selection_loop` identifies the optimal lag (p) and lead (q) lengths. For the specialized Westerlund criterion, the penalty function is defined the same way in R, as explained in Equation 7.

4.1.2 Long-Run Variance Estimation

The method `_lrvar_kernel` implements a Bartlett kernel (HAC) estimator which scales the same way as in the R version explained in Equation 8.

4.1.3 The Error Correction Model (ECM)

The `_get_optimal_model` method then fits the unrestricted ECM regression outlined in Equation 1.

4.1.4 Panel Statistics Construction

The `_run_westerlund_plain` method executes two estimation passes and estimates the group means statistics and panel statistics the same way as outlined in Equations 9 and 11.

4.2 Bootstrap Inference

The `_bootstrap_run` method implements the same Stata-aligned bootstrap algorithm as the R version. Under $H_0 : \alpha_i = 0$, it estimates the restricted short-run model for each unit and performs automatic lag/lead selection on that restricted model when ranges are supplied. Residuals are centered within unit, while ΔX_{it} is centered using all usable observations for the corresponding unit.

Cross-sectional dependence is preserved by resampling actual time clusters jointly across units. The implementation reproduces the `expandc1` 2, clustered time resampling, `newttt/tussent/new` ordering logic, and unit-specific T_i used by the Stata routine. Thus, unbalanced panels do not need to be collapsed to a common bootstrap length. The bootstrap innovations are then constructed according to Equation 13, the autoregressive recursion follows Equation 14, and cumulative sums recover the level series as in Equation 15. As in the R implementation, the Python bootstrap reproduces the original `currlead` compatibility behavior when automatically selected lead orders differ across units.

Each simulated panel is passed back to `_run_westerlund_plain` to obtain G_τ , G_α , P_τ , and P_α . The Python version uses the same finite-sample-corrected bootstrap p -value $(r + 1)/(B + 1)$ as the R version.

4.3 Standardization and Display

Standardized Z-scores are calculated using the same asymptotic moments and deterministic-case logic described in Equation 12. The `run()` method stores these Z-scores and their asymptotic p -values in the returned result bundle rather than limiting them to console display.

4.4 Returned Results and Reporting

The Python result bundle is aligned with the R object and contains the raw statistics, Z-scores, asymptotic and bootstrap p -values, bootstrap distributions, unit-level and detailed individual results, mean-group estimates and standard errors, settings, metadata, and reporting tables. The class exposes `summary()` and `print_summary()` for programmatic and formatted summaries, while `indiv_ecm=True` retains unit-specific ECM tables. Individual OLS tables use residual degrees of freedom and t inference; mean-group reporting uses normal (z) inference, matching the R interface.

Westerlund Test: Bootstrap Distributions

H0: No cointegration | reject when observed statistic is left of the bootstrap critical value

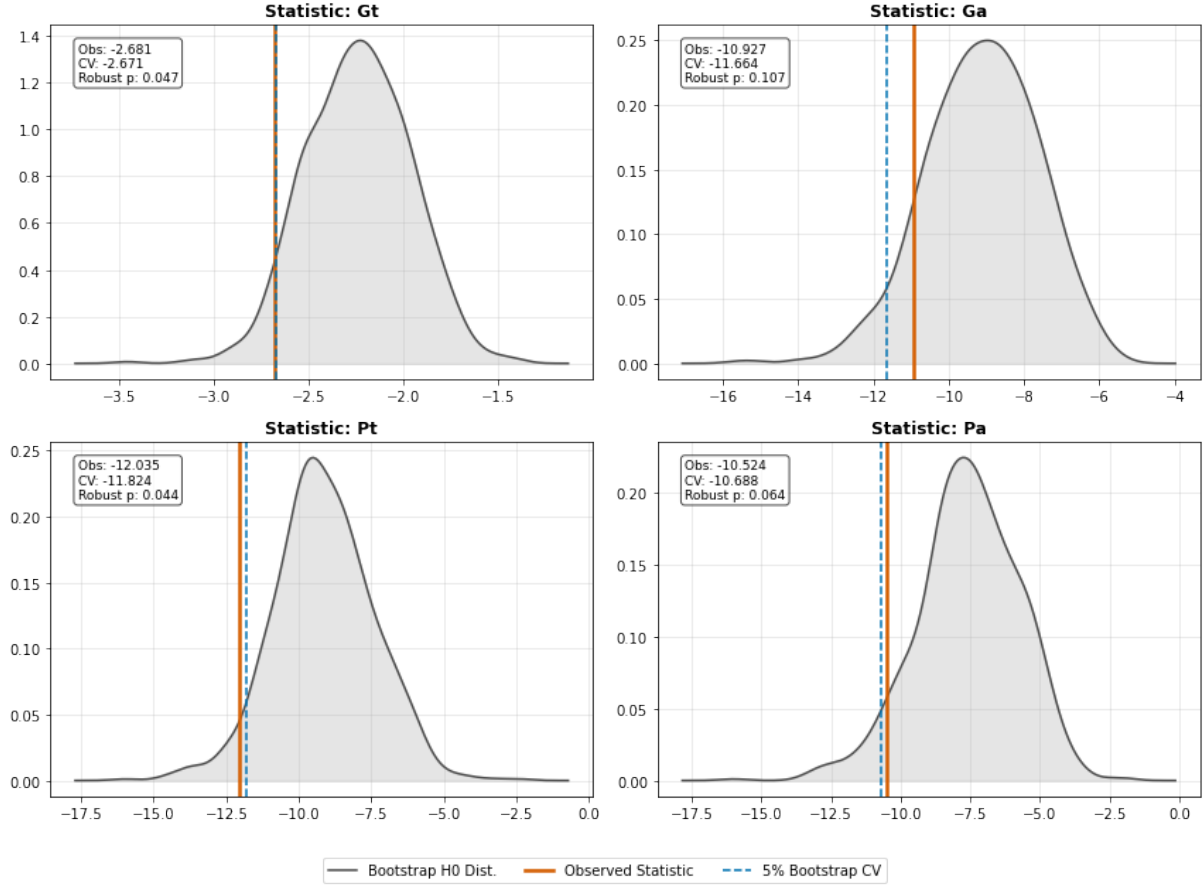


Figure 5: Bootstrapping Distribution: Python Replication of the First Table on p.240 in Persyn and Westerlund (2008).

4.5 Visualization

Similar to its R counterpart, the Python implementation provides `plot()` as an alias for `plot_bootstrap()`. It uses `gaussian_kde` from `scipy.stats` to estimate each empirical bootstrap density and `matplotlib` for display. The critical probability is configurable through `conf_level`; robust p -value annotations can be toggled; and users can configure figure size, colors, line widths, transparency, grid display, output path, and DPI.

Figure 5 provides an example of the visualization result generated by replicating the First Table on p.240 in Persyn and Westerlund (2008).

5 Points to Note

The R and Python implementations are designed to reproduce the same core `xtwest` estimation and bootstrap logic, including restricted-model lag/lead selection, actual-time cluster resampling, unit-specific panel lengths, and the original `currlead` compatibility behavior. Exact numerical identity across R, Python, and Stata should nevertheless not be expected. Non-bootstrap results should agree up to numerical precision when the same data and specification are used. Bootstrap realizations and robust p -values can differ because random-number streams and low-level numerical implementations are software-specific. In addition, the R and Python packages intentionally apply the finite-sample correction $(r + 1)/(B + 1)$ rather than the original `xtwest` reporting rule r/B .

5.1 Randomization

The random-number generators used by R, Python, and Stata do not generally produce identical streams. Both package implementations expose an explicit `seed` argument so that repeated runs are reproducible within the same language, while omitting the seed leaves the current random-number state unconstrained. The R implementation uses `Mersenne-Twister` through `RNGkind`, whereas Python uses `numpy.random.Generator` with the `MT19937` core. Consequently, the same integer seed does not imply replication-by-replication identical draws across software. Cross-software validation should therefore focus on the implemented resampling algorithm and the resulting empirical distribution rather than matching each random draw.

5.2 OLS Equations

The two languages rely on different numerical libraries for solving the OLS systems. R uses `stats::lm`, which is based on QR decomposition with pivoting. The Python implementation explicitly requests QR decomposition for the primary ECM and pooled regressions where parity is most important, although the surrounding model-selection and

numerical-library machinery still differs. Near-collinearity and floating-point conditioning can therefore produce very small cross-software differences even when the specification is identical.

5.3 Floating Point Drifts

Since the levels are built via integration and recursion, tiny differences are magnified.

6 Implementation Examples

In the following, I provide Python and R replications of the results reported by Persyn and Westerlund (2008). For the asymptotic tests, the two implementations should agree up to numerical precision. For bootstrap inference, the same specification and seed are used within each language, but the individual random draws are not expected to be identical across R and Python. Tables 4, 5, and 6 report the R results, while Figure 2 shows the corresponding bootstrap visualization. Tables 1, 2, and 3 report the Python results, with Figure 5 showing the Python bootstrap visualization.

In the R cases, I call:

```
R> library("Westerlund")

R> df_westerlund_test <- haven::read_dta(
+   "Westerlund/Stata/xtwestdata.dta"
+ )

R> table_7 <- westerlund_test(
+   data = df_westerlund_test,
+   yvar = "loghex",
+   xvars = c("loggdgdp"),
+   idvar = "ctr",
+   timevar = "year",
+   constant = TRUE,
+   lags = c(1, 3),
+   leads = c(0, 3),
+   lrwindow = 3,
+   trend = TRUE,
+   westerlund = TRUE
+ )

R> table_p240 <- westerlund_test(
+   data = df_westerlund_test,
```

```

+   yvar = "loghex",
+   xvars = c("loggdp"),
+   idvar = "ctr",
+   timevar = "year",
+   constant = TRUE,
+   lags = 1,
+   leads = 1,
+   bootstrap = 800,
+   seed = 123,
+   lrwindow = 3,
+   trend = TRUE,
+   westerlund = FALSE
+ )
R> table_240_2 <- westerlund_test(
+   data = df_westerlund_test,
+   yvar = "loghex",
+   xvars = c("loggdp"),
+   idvar = "ctr",
+   timevar = "year",
+   constant = TRUE,
+   lags = c(1, 2),
+   leads = c(0, 1),
+   lrwindow = 2,
+   trend = TRUE
+ )
R> plot(table_p240, conf_level = 0.05, show_robust_p = TRUE)

```

In the Python cases, I call:

```

py> from Westerlund import WesterlundTest
py> import pandas as pd
py> df_xtwest = pd.read_stata(
...     'Westerlund/Stata/xtwestdata.dta'
... )
py> table_7 = WesterlundTest(
...     df_xtwest, "loghex", ["loggdp"], "ctr", "year",
...     lags=[1, 3], leads=[0, 3], lrwindow=3,
...     constant=True, trend=True, westerlund=True
... )
py> table_7_results = table_7.run()
py> table_240 = WesterlundTest(
...     df_xtwest, "loghex", ["loggdp"], "ctr", "year",
...     lags=1, leads=1, lrwindow=3, constant=True,
...     trend=True, bootstrap=800, seed=123
... )
py> table_240_results = table_240.run()
py> table_240.plot(conf_level=0.05, show_robust_p=True)
py> table_240_2 = WesterlundTest(
...     df_xtwest, "loghex", ["loggdp"], "ctr", "year",
...     lags=[1, 2], leads=[0, 1], lrwindow=2,
...     constant=True, trend=True
... )
py> table_240_2.run()

```

Table 1: *Python Replication of Table 7 in Westerlund (2007).*

With 20 series and 1 covariate

Average AIC selected lag length: 2.80

Average AIC selected lead length: 1.65

Statistic	Value	Z-value	<i>p</i> -value
Gt	−4.082	−9.613	0.000
Ga	−27.702	−10.626	0.000
Pt	−12.969	−4.100	0.000
Pa	−22.470	−10.119	0.000

Table 2: *Python Replication of the First Table on p.240 in Persyn and Westerlund (2008).***Westerlund ECM Panel Cointegration Tests**

Series: loghex ~ loggdp

N (Groups): 20

Lags: 1-1 | Leads: 1-1 | Window: 3

Statistic	Value	Z-score	<i>p</i> -value	Robust P
Gt	−2.681	−1.734	0.041	0.047
Ga	−10.927	0.714	0.762	0.107
Pt	−12.035	−2.961	0.002	0.044
Pa	−10.524	−1.160	0.123	0.064

Table 3: *Python Replication of the second table on p.240 in Persyn and Westerlund (2008).***Westerlund ECM Panel Cointegration Tests**

Series: loghex ~ loggdp

N (Groups): 20

Lags: 1-2 | Leads: 0-1 | Window: 2

Statistic	Value	Z-score	<i>p</i> -value	Robust P
Gt	−2.736	−2.036	0.021	—
Ga	−11.253	0.499	0.691	—
Pt	−12.859	−3.903	0.000	—
Pa	−11.773	−2.072	0.019	—

Table 4: *R Replication of Table 7 in Westerlund (2007).*

With 20 series and 1 covariate

Average AIC selected lag length: 2.80

Average AIC selected lead length: 1.65

Statistic	Value	Z-value	<i>p</i> -value
Gt	−4.082	−9.613	0.000
Ga	−27.702	−10.626	0.000
Pt	−12.969	−4.100	0.000
Pa	−22.470	−10.119	0.000

Table 5: *R* Replication of the First Table on p.240 in Persyn and Westerlund (2008).

Westerlund ECM Panel Cointegration Tests				
Series: loghex ~ loggdp				
N (Groups): 20				
Lags: 1-1 Leads: 1-1 Window: 3				
Statistic	Value	Z-score	<i>p</i> -value	Robust P
Gt	−2.681	−1.731	0.042	0.070
Ga	−10.927	0.713	0.762	0.125
Pt	−12.035	−2.959	0.002	0.051
Pa	−10.524	−1.160	0.123	0.056

Table 6: *R* Replication of the second table on p.240 in Persyn and Westerlund (2008)

Westerlund ECM Panel Cointegration Tests				
Series: loghex ~ loggdp				
N (Groups): 20				
Lags: 1-2 Leads: 0-1 Window: 2				
Statistic	Value	Z-score	<i>p</i> -value	Robust P
Gt	−2.736	−2.033	0.021	–
Ga	−11.253	0.499	0.691	–
Pt	−12.859	−3.902	0.000	–
Pa	−11.773	−2.072	0.019	–

7 Conclusion

The **Westerlund** package provides aligned Python and R implementations of the Westerlund panel cointegration tests for users who prefer open-source workflows. The core estimation and bootstrap procedures are designed to follow the original **xtwest** logic closely, while both language versions enrich the interface with reusable result objects, unit-level and mean-group output, summary and reporting methods, explicit seed control, and configurable bootstrap visualization.

Meanwhile, the current **Westerlund** implementations face the same limitation of the **xtwest** function: they can only address cases with at most six regressors due to their reliance on a hard-coded table. Future work could explore allowing users to simulate the moment to enhance the precision of estimations and allow the use of more regressors. However, as the Westerlund test has a high parameter density which makes it power-hungry and macro panel data often suffers from a small *N* problem, use cases involving

more than six regressors are probably less likely. Other pathways of future work would be to provide users with options to choose whether to apply finite sample correction, estimate the runtime, etc.

8 Bibliography

References

- Croissant, Y., Millo, G., & Tappe, K. (2006, June). Plm: Linear Models for Panel Data [Institution: Comprehensive R Archive Network Pages: 2.6-7]. <https://doi.org/10.32614/CRAN.package.plm>
- Hadri, K. (2000). Testing for stationarity in heterogeneous panel data. *The Econometrics Journal*, 3(2), 148–161. <https://doi.org/10.1111/1368-423X.00043>
- Im, K. S., Pesaran, M., & Shin, Y. (2003). Testing for unit roots in heterogeneous panels. *Journal of Econometrics*, 115(1), 53–74. [https://doi.org/10.1016/S0304-4076\(03\)00092-7](https://doi.org/10.1016/S0304-4076(03)00092-7)
- Levin, A., Lin, C.-F., & James Chu, C.-S. (2002). Unit root tests in panel data: Asymptotic and finite-sample properties. *Journal of Econometrics*, 108(1), 1–24. [https://doi.org/10.1016/S0304-4076\(01\)00098-7](https://doi.org/10.1016/S0304-4076(01)00098-7)
- Natsiopoulos, K., & Tzeremes, N. (2020, April). ARDL: ARDL, ECM and Bounds-Test for Cointegration [Institution: Comprehensive R Archive Network Pages: 0.2.4]. <https://doi.org/10.32614/CRAN.package.ARD>
- Newey, W. K., & West, K. D. (1994). Automatic Lag Selection in Covariance Matrix Estimation. *The Review of Economic Studies*, 61(4), 631–653. <https://doi.org/10.2307/2297912>
- Pedroni, P. (2004). Panel cointegration: Asymptotic and finite sample properties of pooled time series tests with an application to the ppp hypothesis. *Econometric Theory*, 20(03). <https://doi.org/10.1017/S0266466604203073>
- Persyn, D., & Westerlund, J. (2008). Error-Correction-Based Cointegration Tests for Panel Data. *The Stata Journal: Promoting communications on statistics and Stata*, 8(2), 232–241. <https://doi.org/10.1177/1536867X0800800205>
- Pesaran, M. H. (2007). A simple panel unit root test in the presence of cross-section dependence. *Journal of Applied Econometrics*, 22(2), 265–312. <https://doi.org/10.1002/jae.951>

- Pesaran, M. H. (2021). General diagnostic tests for cross-sectional dependence in panels. *Empirical Economics*, 60(1), 13–50. <https://doi.org/10.1007/s00181-020-01875-7>
- Pesaran, M. H., & Shin, Y. (1999). An Autoregressive Distributed-Lag Modelling Approach to Cointegration Analysis. In S. Strom (Ed.), *Econometrics and Economic Theory in the 20th Century* (pp. 371–413). Cambridge University Press. <https://doi.org/10.1017/CCOL521633230.011>
- Pesaran, M. H., Shin, Y., & Smith, R. J. (2001). Bounds testing approaches to the analysis of level relationships. *Journal of Applied Econometrics*, 16(3), 289–326. <https://doi.org/10.1002/jae.616>
- Westerlund, J. (2007). Testing for Error Correction in Panel Data*. *Oxford Bulletin of Economics and Statistics*, 69(6), 709–748. <https://doi.org/10.1111/j.1468-0084.2007.00477.x>
- Zientara, P., & Kujawski, L. (2017, December). PooledMeanGroup: Pooled Mean Group Estimation of Dynamic Heterogenous Panels [Institution: Comprehensive R Archive Network Pages: 1.0]. <https://doi.org/10.32614/CRAN.package.PooledMeanGroup>